

1 Testing & infrastructure

This section describes the infrastructure a new engineer inherits at the pinned commit [254cd41](#): six GoogleTest binaries registered with ctest as 246 entries, a small family of oracle patterns that the suites use instead of golden values, a committed fixture corpus the tests locate through compile-time absolute paths, a Clang-only [llvm-cov](#) coverage pipeline living in its own build tree, and one continuous integration (CI) workflow that drives five operating systems through the same CMake presets developers use locally (?? introduced the presets and the warning policy; this section covers the rest). The infrastructure is disciplined, but it has sharp edges, and each is marked where it lives: a coverage target that predates the newest test suite, one stale comment claiming tooling that does not exist, and an install rule that would ship the vendored dependencies alongside the application.

1.1 Six binaries, two registrations

Test sources live next to what they test: `model/tests/` and `sim/tests/` hold the per-layer unit suites, and the top-level `tests/` directory holds everything that crosses layers — integration, propagator behavior, and the two suites that drive the command-line interface (CLI). [Table 1](#) lists the six binaries with their live per-case counts.

Binary	Cases	What it exercises	Defined at
<code>model_tests</code>	144	part composition and inertia, motors and thrust curves (with the RASP parser), the concrete part types, Barrowman aero, the part factory, placement types, the resolver sweep, the diagnostics gate	<code>model/tests/CMakeLists.txt:2</code>
<code>sim_tests</code>	27	the atmosphere models, <code>Environment</code> , the RK45 solver, spherical geoid and gravity, integrator selection	<code>sim/tests/CMakeLists.txt:2</code>
<code>integration_tests</code>	43	end-to-end physics through the controller, motor-database and design persistence, the placement-invariance gate, the poke-through regression	<code>tests/CMakeLists.txt:4</code>
<code>propagator_tests</code>	6	the propagator’s termination guards and trajectory statistics, driven by a mock <code>Propagatable</code>	<code>tests/CMakeLists.txt:32</code>
<code>cli_tests</code>	3	a scripted REPL build → save → reload → fly session	<code>tests/CMakeLists.txt:48</code>
<code>design_matrix_tests</code>	16	the committed fixture matrix, round-trip fidelity, and the 1/4A → M flight ladder	<code>tests/CMakeLists.txt:69</code>

Table 1. The six test binaries at the pinned commit, with per-case counts taken live from each binary’s `--gtest_list_tests` output (144 + 27 + 43 + 6 + 3 + 16 = 239 cases across 49 distinct gtest suites).

The link lines mirror the layering of ??: each binary links exactly the topmost layer it exercises. `model_tests` links `model` and `utils` `model/tests/CMakeLists.txt:22`; `sim_tests` links `sim` `sim/tests/CMakeLists.txt:12`; `propagator_tests` links `model` — which transitively pulls in `sim` and `utils` and supplies the out-of-line `Propagatable` constructor symbol `tests/CMakeLists.txt:29`; `integration_tests` links `qtrocket_core`, the same controller archive the executables link `tests/CMakeLists.txt:19`; and

the two CLI-driving suites link the `cli` library, so they exercise the identical `Repl` object code users type into `tests/CMakeLists.txt:56`, `tests/CMakeLists.txt:78`.

Every binary is registered with `ctest` twice. First, per case: `gtest_discover_tests` runs each binary at build time and registers every `Suite.Case` as its own `ctest` entry `model/tests/CMakeLists.txt:28`, `sim/tests/CMakeLists.txt:17`, `tests/CMakeLists.txt:24` — 239 entries on this tree, which is what makes per-case filtering like `ctest -R PartTests` work. Second, in aggregate: one `add_test(NAME qtrocket_<suite>_tests ...)` per binary runs the whole executable as a single `ctest` entry `model/tests/CMakeLists.txt:31`, `sim/tests/CMakeLists.txt:20`, `tests/CMakeLists.txt:27`. Seven aggregates exist for six binaries — the design-matrix binary registers twice more, as [Section 1.3](#) explains — for a total of $239 + 7 = 246$ registered tests, counted live with `ctest -N` at the pinned commit.

Aggregate names are the selection contract

Both CI and the coverage pipeline run *only* the aggregates: CI filters with `ctest -R 'qtrocket_*` `.github/workflows/cmake-multi-platform.yml:74` and the coverage script defaults to `^qtrocket_.*tests$` `cmake/RunLlvmCoverage.cmake:13`. Note that `'qtrocket_*` is a regular expression, not a glob — it means “qtrocket followed by zero or more underscores, anywhere in the name” — and it selects exactly the seven aggregates only because no discovered `Suite.Case` name contains the substring `qtrocket`. The corollary binds new work: a future suite that forgets to register a `qtrocket_`-prefixed aggregate silently drops out of both CI and coverage, with no error anywhere.

The net has known holes, stated here and inventoried in ??: no test binary covers `gui/` or `visualizer/` (the visualizer is also the only project target without a `qtrocket_enable_coverage` call — `visualizer/CMakeLists.txt` contains none, where every other target has one); and `utils/` has no test directory of its own `utils/CMakeLists.txt`, so `CurlConnection` is never executed by any suite. Each of these is an absence verified by searching the tree, not an impression.

1.2 Oracle patterns

The suites share a house style: assertions state *properties that must hold* — closed-form values derived independently, invariants, inequalities — rather than golden numbers snapshotted from the code under test. Four patterns recur, and each is worth learning by example because new tests are expected to follow them.

Independent numeric oracles. The inertia-tensor suite states the rule in its header: the tests pin the per-unit-mass helpers “against INDEPENDENT numeric oracles (disk / surface / volume-mesh integration) — never against the closed form they validate” `model/tests/InertiaTensorsTests.cpp:8`. The solid cone is checked against a Riemann stack of 400,000 thin disks `model/tests/InertiaTensorsTests.cpp:28`, the conical shell against a stack of hoops `model/tests/InertiaTensorsTests.cpp:55`, and the trapezoidal fin set against a brute-force volume mesh over N prisms `model/tests/InertiaTensorsTests.cpp:92` — the stated acceptance gate for the fin set’s closed-form algebra. Agreement is demanded at 10^{-5} relative for the cone `model/tests/InertiaTensorsTests.cpp:157` and 0.5% for the mesh `model/tests/InertiaTensorsTests.cpp:209`; the textbook closed forms are then additionally pinned exactly, e.g. $I_{zz} = \frac{3}{10}R^2$ per unit mass at 10^{-15} `model/tests/InertiaTensorsTests.cpp:161`. The logic is the point: a closed form checked against its own algebra proves nothing, but an independent discretization of the defining integral converges to the same number only if both are right. (The tensors themselves are the subject of ??.)

Literal-anchor regression pins. Where a bug was found and fixed, its exact numbers are frozen into an assertion with the derivation written beside it. The poke-through regression loads the one deliberately self-intersecting fixture through the real file reader and pins the diagnostic's location and depth: `EXPECT_NEAR(d.zWorld, -0.26, 1e-9)` and penetration $0.0376 - 0.0395$ ($0.26/0.30$) ≈ 0.003367 m `tests/PokeThroughRegressionTests.cpp:63`, then asserts that the composite gate refuses the failed solve — `EXPECT_THROW` on `getCompositeI` `tests/PokeThroughRegressionTests.cpp:67` — the fail-closed posture of `??`, exercised end to end. The same file pins every resolved station of the whitepaper's worked example to hand-computed literals `tests/PokeThroughRegressionTests.cpp:70`. The invariance suite carries two more: the solid nose cone's local center of mass (CM) hand-pinned to -0.225 m at 10^{-12} , with the $\bar{h} - L = -3L/4$ derivation in the comment `tests/PlacementInvarianceTests.cpp:445`, and the fin-set CM recomputed from the fixture's raw trapezoid geometry rather than read back through the part `tests/PlacementInvarianceTests.cpp:458`. Each pin defends itself against fixture drift — a guard assertion such as “fixture cone length changed — re-derive the pin” fails first if someone edits the geometry `tests/PlacementInvarianceTests.cpp:450`.

Invariance gates. The placement migration (`??`) rests on the claim that it changed no physics, and the gate makes that claim falsifiable. A frozen baseline snapshots composite mass, CM, inertia, and every part's resolved station for the 24-fixture corpus `tests/PlacementInvarianceTests.cpp:66`, formatted at `%.17g` so every double round-trips to the identical IEEE-754 value `tests/PlacementInvarianceTests.cpp:88`. The live tree must reproduce it: mass and inertia directly `tests/PlacementInvarianceTests.cpp:266`, CM and stations after the one known tip-datum shift `tests/PlacementInvarianceTests.cpp:286`, `tests/PlacementInvarianceTests.cpp:299`, all within 10^{-9} — a tolerance sized to admit the ULP-level drift of re-associated floating-point arithmetic and nothing larger `tests/PlacementInvarianceTests.cpp:151`, with a diagnostic test reporting the worst observed drift so the tolerance can be judged against reality `tests/PlacementInvarianceTests.cpp:328`. The baseline file declares its own epistemics: never regenerate it to make a failing migration pass, because “a diff here means the refactor changed the physics” `tests/data/placement-invariance-baseline.txt:4`. One quantity is deliberately excluded: the center of pressure, because the migration *corrected* a CM-contaminated legacy value, so baseline conformance there would pin the bug `tests/PlacementInvarianceTests.cpp:318`.

End-to-end flight sweeps. The design-matrix suite flies the whole engine — reader, resolver, motor database, propagator, both integrators — across seven reference airframes and asserts physical inequalities rather than trajectories. Per motor flown: apogee finite, above 1 m (“near-zero apogee implies a $t=0$ abort, not real flight” `tests/DesignMatrixTests.cpp:468`), below 10^6 m `tests/DesignMatrixTests.cpp:469`, and strictly increasing along the motor ladder — `EXPECT_GT(ap, prev)` `tests/DesignMatrixTests.cpp:470`. The atmosphere sweep flies each motor twice from identical state — vacuum with `setdrag 0`, then “US Standard 1976” with `setdrag 0.75` `tests/DesignMatrixTests.cpp:542` — and requires `EXPECT_LT(drag, vac)` `tests/DesignMatrixTests.cpp:552`: drag dissipates energy, so apogee under drag must be strictly lower. The two integrators of `??` must agree in vacuum to 2%: `EXPECT_NEAR(rk45, rk4, 0.02 * rk4)` `tests/DesignMatrixTests.cpp:599`. Launch-pad behavior is pinned from both sides: the smallest motor in the corpus must lift the lightest airframe nominally `tests/DesignMatrixTests.cpp:610`, while a 0.59 N s motor under a 2000 kg/m³ tube must produce a “no liftoff” warning and an apogee below 1 m `tests/DesignMatrixTests.cpp:630`. A units regression rides along: online-sourced motor mass must arrive in kilograms, not grams `tests/DesignMatrixTests.cpp:269` — the fix for a real bug in which a 6.1-gram motor loaded as 6.1 kilograms (`??`). These oracles survive legitimate physics improvements — a better drag model still satisfies every inequality — but a

sign error, a unit slip, or a broken ignition path fails them immediately.

Impulse classes and the motor ladder

Model-rocket motors are graded by total impulse $I_{\text{tot}} = \int F(t) dt$ (newton-seconds) into letter classes, each class doubling the ceiling of the one before: A up to 2.5 Ns, B up to 5, C up to 10, and so on up to M at 10240 Ns; fractional classes 1/4A and 1/2A sit below A. This is the standard NAR/TRA convention — domain background, not a table encoded in the repository. A “ladder” is a set of motors of one diameter arranged in ascending total impulse. For a fixed airframe in vacuum under fixed gravity, more delivered impulse means more kinetic energy at burnout and therefore a higher apogee, which is why apogee monotonicity along the ladder is a coarse but robust integral check on the entire motor $\rightarrow$ force $\rightarrow$ integrator chain.

1.3 The light/heavy registration split

The ladder sweeps are expensive by nature: they “fly every motor of each class’s full 1/4A $\rightarrow$ M ladder, in vacuum and through the atmosphere — with high-impulse motors coasting to tens of km this dominates the runtime” `tests/CMakeLists.txt:85`. The build resolves the tension between coverage and iteration speed by registering the same binary a third time ([Listing 1](#)): a default (“light”) aggregate that runs all 16 cases, and a “heavy” aggregate that re-runs only the sweep cases — the `--gtest_filter=FlightMatrix.*:Atmosphere.*` filter selects 3 of the 16 — tagged with the ctest label `heavy` and the environment variable `QTROCKET_FULL_LADDER=1` `tests/CMakeLists.txt:96`.

```
add_test(NAME qtrocket_design_matrix_tests COMMAND design_matrix_tests)

add_test(NAME qtrocket_design_matrix_heavy_tests
  COMMAND design_matrix_tests "--gtest_filter=FlightMatrix.*:Atmosphere.*")
set_tests_properties(qtrocket_design_matrix_heavy_tests PROPERTIES
  LABELS heavy
  ENVIRONMENT "QTROCKET_FULL_LADDER=1")
```

Listing 1. The double registration of the design-matrix binary `tests/CMakeLists.txt:94`. The light entry runs the whole suite with one motor per class; the heavy entry re-runs just the two sweep suites under the full-ladder environment variable, labeled so ctest can include or exclude it by cost.

The switch lives in the test code, not the build ([Listing 2](#)): each of the seven `Ladder` entries pairs a reference airframe with its motors in ascending total impulse `tests/DesignMatrixTests.cpp:349`, and `motorsToFly` returns either the full ladder or just its smallest rung:

$$\text{motorsToFly}(\text{lad}) = \begin{cases} \text{lad.motors} & \text{if QTROCKET_FULL_LADDER is set,} \\ \{ \text{lad.motors.front()} \} & \text{otherwise.} \end{cases}$$

The check is `std::getenv(...) != nullptr` `tests/DesignMatrixTests.cpp:377` — presence-only, so even `QTROCKET_FULL_LADDER=0` enables the full ladder. The light run keeps the coverage *topology* (every diameter class, every airframe, both integrators) while flying only one motor per class; under it the monotonicity loop degenerates to a single motor, so only the heavy entry actually exercises the apogee-ordering oracle `tests/DesignMatrixTests.cpp:461`.

The selection algebra falls out of the label, and the build file spells it out `tests/CMakeLists.txt:92`: `ctest -R [qtrocket_*]` runs both entries (full coverage — 7 tests); `ctest -LE heavy` is the fast development loop (everything except the full-ladder sweeps — 245 of the 246 entries); `ctest -L`

```
std::vector<std::string> motorsToFly(const Ladder& lad)
{
    if(std::getenv("QTROCKET_FULL_LADDER") != nullptr)
        return lad.motors;
    return { lad.motors.front() };
}
```

Listing 2. The entire light/heavy switch `tests/DesignMatrixTests.cpp:375`. The build’s heavy ctest entry sets the environment variable; the code reads only its presence.

heavy runs only the sweeps (exactly 1 entry). Because the heavy entry is a strict re-run subset — three cases under a stronger environment — the plain aggregate filter gets full coverage without paying full-ladder cost on the thirteen cheap cases twice.

1.4 Test data and fixtures

Tests locate their committed data through two compile-time definitions: `QTROCKET_DATA_DIR` points at the bundled motor data in `data/` and `QTROCKET_TEST_DATA_DIR` at the fixture corpus in `tests/data/`, both expanded to absolute paths from `CMAKE_SOURCE_DIR` `tests/CMakeLists.txt:15`, `tests/CMakeLists.txt:53`, `tests/CMakeLists.txt:74`.

Tests are cwd-independent; shipped binaries carry no source paths

The rationale is stated at the definition site: “Absolute paths so the tests run from any cwd” `tests/CMakeLists.txt:13` — a test binary invoked by ctest, an IDE, or by hand from any directory finds the same fixtures. The definitions are `PRIVATE` to the test targets, and a search across `model/`, `sim/`, `cli/`, and `gui/` finds no other consumer of either macro — so the executables users run embed no build-machine paths, and the application starts with an empty motor database (the bundled `data/Aerotech.rse` is test input, not runtime data; ??).

The corpus itself is small and self-describing `tests/data/README.md: 25` .qrd designs under `tests/data/designs/` — the 24-fixture invariance corpus spanning seven motor-diameter classes (13, 18, 24, 29, 38, 54, 75 mm, each airframe sized so its motor fits the body bore), plus `xl75_multi_pokethrough.qrd`, the deliberately self-intersecting offender — a motor-database fixture `motors/estes_small.qmd`, and the frozen invariance baseline. The corpus checks itself: `designFiles()` enumerates the directory at runtime, excluding the poke-through offender `tests/DesignMatrixTests.cpp:329`, and a dedicated test pins the result against the 24-name expected list — failing with “fixture count drifted from the documented matrix” if anyone adds or removes a design without updating the manifest `tests/DesignMatrixTests.cpp:393`.

1.5 Coverage

Coverage is Clang/LLVM source-based coverage, deliberately isolated from normal development. The `coverage-clang` preset inherits `debug-clang` but redirects to a separate `build-coverage/` tree and sets `QTROCKET_ENABLE_COVERAGE=ON` `CMakePresets.json:32`, so instrumented objects never pollute `build/`. The option is hard-gated to Clang: any other compiler hits a message(`FATAL_ERROR` . . .) naming the reason — the pipeline uses `llvm-cov`, whose instrumentation format GCC does not emit `CMakeLists.txt:101`. GCC remains fully supported for building and testing through the `debug-gcc/release-gcc` presets; it is only coverage that requires Clang. Tool discovery tolerates distribution-versioned binary names (`llvm-cov-21` down to `-17`) `CMakeLists.txt:105`.

Instrumentation is applied per target by one helper, `qtrocket_enable_coverage` `CMakeLists.txt:118`: a no-op unless the option is on, it adds the compile flags (`-O0 -g -fprofile-instr-generate -fcoverage-mapping` `CMakeLists.txt:108`) and — only when the target is not a static library — the matching link flags `CMakeLists.txt:121`. The distinction is load-bearing: static archives never see a link line, so the profiling runtime is linked once into each final test executable rather than uselessly requested on every archive.

The `coverage` target itself `CMakeLists.txt:256` runs the whole pipeline through a script-mode CMake driver, `cmake/RunLlvmCoverage.cmake`. In order: `ctest` runs the aggregate suites under `L_VLM_PROFILE_FILE=<raw>/%p-%m.pro` `cmake/RunLlvmCoverage.cmake:33` — `%p` (process id) and `%m` (binary signature) give each test process its own profile file instead of clobbering one — and a nonzero `ctest` result is fatal `cmake/RunLlvmCoverage.cmake:39`: the pipeline is fail-closed, refusing to report coverage of a failing tree. The raw profiles are merged with `llvm-profdata`, `merge -sparse` `cmake/RunLlvmCoverage.cmake:50`; `llvm-cov report` writes the text summary to `build-coverage/coverage/summary.txt` `cmake/RunLlvmCoverage.cmake:80`; and `llvm-cov show` emits the HTML report with per-region line counts and branch counts to `build-coverage/coverage/html/` `cmake/RunLlvmCoverage.cmake:98`. The ignore regex excludes `_deps`, the build trees, `tests/` itself, and the vendored `gui/qcustomplot.cpp` `CMakeLists.txt:262`, so the percentages measure project code exercised by the suites — not dependency code and not the tests' own bodies.

```
set(QTROCKET_COVERAGE_BINARIES
  "$<TARGET_FILE:sim_tests>"
  "$<TARGET_FILE:model_tests>"
  "$<TARGET_FILE:integration_tests>"
  "$<TARGET_FILE:propagator_tests>"
  "$<TARGET_FILE:cli_tests>")
```

Listing 3. The coverage target's binary list `CMakeLists.txt:248` — five binaries, not six. The same five names appear in the target's `DEPENDS` `CMakeLists.txt:267`, while the test-selection regex matches all seven aggregates `CMakeLists.txt:261`.

The coverage target predates the design-matrix suite

A verified latent defect: `design_matrix_tests` is missing from both the `-object` list and the `DEPENDS` of the `coverage` target ([Listing 3](#)), yet the selection regex `^qtrocket_.*tests$` `CMakeLists.txt:261` matches both of that binary's aggregate entries — the coverage stack landed in one commit (`2c9404b`) before the design-matrix suite existed, and the lists were never updated. Three consequences follow. In a fresh coverage tree, the target does not build the binary, the inner `ctest` run hits an unbuilt executable, and the script aborts at its fail-closed gate `cmake/RunLlvmCoverage.cmake:39` — unless the user has already built the whole tree. When the binary is built, every coverage run pays for the heavy full-ladder sweep, which is slow by design `tests/CMakeLists.txt:85`. And although the binary is instrumented `tests/CMakeLists.txt:72` and its profiles are merged, it is never passed to `llvm-cov` as an `-object`, so code exercised *only* by it would be invisible in the report — benign today because it links the same `cli` and `qtrocket_core` archives whose mappings arrive through the other binaries, and its own translation unit sits under the ignored `tests/` path.

1.6 Continuous integration

CI is one GitHub Actions workflow, triggered on pushes and pull requests against `development` `.github/workflows/cmake-multi-platform.yml:12`. Its design statement leads the file: CI drives the build through the project’s presets, “so it exercises the same Ninja-based configure/build/test path that developers use locally instead of each platform’s default generator” `.github/workflows/cmake-multi-platform.yml:3`. There is no separate CI build system to drift out of sync — [Table 2](#) is simply the preset table of ?? mapped onto runners.

Runner	Preset	Toolchain
ubuntu-latest	release-gcc	GCC
ubuntu-latest	release-clang	Clang
macos-latest	release-clang	AppleClang
windows-latest	release-msvc	MSVC (Ninja + cl)
ubuntu-latest (QEMU VM)	release-clang	FreeBSD 15.0, Clang/libc++

Table 2. The CI matrix `.github/workflows/cmake-multi-platform.yml:28`: four hosted-runner entries with `fail-fast` disabled — so one platform’s breakage does not mask the others — plus the separate FreeBSD virtual-machine job (lines 26 and 94 of the same workflow).

The matrix steps are few: install CMake and Ninja (not preinstalled on the Linux/macOS images, and every preset requires Ninja) `.github/workflows/cmake-multi-platform.yml:37`; install Qt 6.10.2 `.github/workflows/cmake-multi-platform.yml:42`; on Windows, set up the MSVC toolchain environment explicitly, because the Ninja generator does not run `vcvars` the way the Visual Studio generator implicitly did `.github/workflows/cmake-multi-platform.yml:48`; warm a ccache keyed on OS × preset, explicitly to cache the compiled FetchContent dependencies across runs `.github/workflows/cmake-multi-platform.yml:54`; then configure, build, and test through the presets `.github/workflows/cmake-multi-platform.yml:61`. The test step’s filter is the aggregate contract of [Section 1.1](#) — and because *both* design-matrix entries match it, every push and pull request runs the heavy full-ladder sweep in CI `.github/workflows/cmake-multi-platform.yml:74`. (The comment above that step still names only four suites — “(model, sim, integration, propagator)” `.github/workflows/cmake-multi-platform.yml:72`; it predates the CLI and design-matrix suites. Read the filter, not the comment.)

The fifth platform has no hosted runner, so the workflow boots a FreeBSD 15.0 virtual machine (QEMU, via `vmactions/freebsd-vm`) inside an `ubuntu-latest` runner `.github/workflows/cmake-multi-platform.yml:76`. The job is separate from the matrix because the toolchain setup differs: Qt comes from `pkg` and the base compiler is Clang with `libc++`, but the build itself reuses the `release-clang` preset unchanged — `pkg install cmake ninja git pkgconf qt6-base qt6-tools`, then the same three preset commands under `set -e` `.github/workflows/cmake-multi-platform.yml:101`. The hermetic curl configuration of ?? pays off exactly here, and the workflow says so: “The hermetic curl config means OpenSSL + zlib (both in the FreeBSD base system) are the only external libs curl needs” `.github/workflows/cmake-multi-platform.yml:99`. A 90-minute timeout bounds the run, since the VM has no ccache and compiles every FetchContent dependency from source `.github/workflows/cmake-multi-platform.yml:87`.

What CI does not run, verified by the single workflow file being the only one under `.github/workflows/`: no coverage job, no sanitizer job, no debug-preset job, no artifact upload, and no release or packaging step. ?? carries the inventory.

1.7 Warning hygiene

?? stated the policy — `-Wall -Wextra -Wpedantic -Werror` on GCC and Clang `CMakeLists.txt:97`, `/W4 /WX` on MSVC `CMakeLists.txt:88` — and the detail that makes it livable is placement. The `add_compile_options` calls are directory-scoped and sit *after* all five `FetchContent_MakeAvailable` blocks but *before* the project’s own subdirectories and targets, so vendored dependencies compile with their own flags while every project target must be warning-clean; the history note records that the exemption was verified in `compile_commands.json` when the flags landed `TODO.md:23`. One vendored file lives inside a project target and is exempted per source: the third-party plotting widget `gui/qcustomplot.cpp` is compiled with `-w` — “qcustomplot throws some warnings, so silence them so we can turn on `-Werror` for our code” `CMakeLists.txt:152`.

On MSVC, `/WX` turns the compiler’s C4996 “use the `_s` variant” deprecation nags into hard errors, so the build defines `_CRT_SECURE_NO_WARNINGS` tree-wide, with the rationale recorded in place: those deprecations are non-standard editorializing against standard C++ functions like `std::getenv`, genuine `[[deprecated]]` uses still warn, and GCC/Clang remain the real safety net `CMakeLists.txt:89`. One line of that comment block has gone stale and should not be trusted: “We also run ASAN and clang-tidy for additional security” `CMakeLists.txt:93`. No `-fsanitize` flag, no `CLANG_TIDY` property, and no `.clang-tidy` configuration exists anywhere in the project’s CMake, presets, or CI — each a failed search of the tree. The warning flags are real and enforced; the sanitizer claim is a comment about infrastructure that does not exist.

1.8 What the infrastructure does not do

Distribution is where the infrastructure stops. The project contains exactly one `install()` rule, for the GUI executable alone — `install(TARGETS qtrocket BUNDLE DESTINATION . LIBRARY DESTINATION ...)` `CMakeLists.txt:308`. Nothing installs `qtrocket-cli`, `qtrocket-visualizer`, the motor data, or any header, and the one rule works on Linux only through side effects: it names no `RUNTIME DESTINATION`, so the executable lands in `bin/` by CMake’s implicit default, and `CMAKE_INSTALL_LIBDIR` is defined only because some dependency’s build includes `GNUInstallDirs` — no project CMake file includes it (a failed search). There is no CPack configuration anywhere in the project’s CMake files, hence no installers or packages of any kind.

`cmake --install` would ship the vendored dependencies

None of the five `FetchContent_Declare` blocks passes `EXCLUDE_FROM_ALL` `CMakeLists.txt:15`, so the generated root install script includes the `_deps` sub-installs for GoogleTest, jsoncpp, curl, Eigen, and Boost alongside the project’s own rule — observable in the generated `build/cmake_install.cmake`. Running `cmake --install build` today would install GoogleTest, curl, and Boost headers and archives into the prefix next to `qtrocket`. The install rule is a template stub, not a distribution story; treat it accordingly.

Internationalization is dead scaffolding from the same template: the build requires `LinguistTools` `CMakeLists.txt:139`, lists the single translation source `qtrocket_en_US.ts` `CMakeLists.txt:145` among the GUI sources `CMakeLists.txt:187`, and runs `qt_create_translation(QM_FILES ...)` `CMakeLists.txt:201` — but `QM_FILES` appears nowhere else in any CMake file, so the compiled catalogs are never attached to a target or installed; the roadmap parks translations explicitly `TODO.md:79`. The macOS bundle identifier is likewise still the template placeholder `my.example.com` `CMakeLists.txt:301`.

The shape of the whole is easy to state. Everything between a developer’s editor and a green five-platform CI run is built, pinned, and self-checking: six suites whose assertions are oracles, a

fixture corpus that audits its own manifest, a coverage pipeline that refuses to report on a failing tree, and a warning wall that vendored code cannot breach. Everything between that green run and a user's machine — installers, packages, translations, a runtime motor database — is not built yet, and ?? inventories it with the rest of the unfinished work.